Lab 09 — Questions

Question 1 [Understanding]

A colleague has a thirty-line `start.sh` of `podman network create`, `podman volume create` and `podman run`, and a `stop.sh` that does the reverse. "It works, why Compose?" Give three things that `podman compose up` / `down` know and their scripts do not, without mentioning length.

Question 2 [Diagnosis]

The `.env` file next to `compose.yaml` contains `APP_MESSAGE=hello from dotenv`. After `podman compose up -d`, `curl localhost:18080/api/` still returns the default message. Yet `podman compose config` reports no error. Explain what `.env` really does, and where `APP_MESSAGE` should have been declared.

Question 3 [Prediction]

Two versions of the dependency of `api` on `db`:

```
depends_on: [db]
```

```
depends_on:
  db:
    condition: service_healthy
```

For a real Spring Boot API, describe what happens at `up -d` in each case (order, logs, final state), and why the second form requires a `healthcheck` block on `db`.

Question 4 [Diagnosis]

```
api:
  image: localhost/api:1.0
  healthcheck:
    test: ["CMD", "curl", "-f", "http://localhost:8080/actuator/health"]
    interval: 5s
    retries: 3
```

After thirty seconds, `podman compose ps` shows `api` as `(unhealthy)` and `web` never started, although `curl localhost:18080` worked before the healthcheck was added. Diagnose with one command, fix it, and say why the image is to blame.

Question 5 [Analysis]

In `compose.yaml`, compare `- pgdata:/var/lib/postgresql/data` (with `volumes: pgdata:` at the bottom) and `- ./data:/var/lib/postgresql/data`: who creates the directory, who owns the files under rootless, what does `down -v` do in each case, and which one do you pick for the stack's database?

Question 6 [Prediction]

A developer clones the project into `~/work/stack-v2/` instead of `~/work/stack/`, runs `podman compose up -d`, and finds an **empty** database: "Compose lost my data!" The file has no `name:`

key. Explain, show with `podman volume ls` what happened, and give two ways to avoid the problem.

Question 7 [Diagnosis]

```
$ podman compose -f compose.yaml -f compose.prod.yaml up -d
$ podman compose exec api env | grep APP_PROFILE
APP_PROFILE=dev
$ podman exec stack_api_1 env | grep APP_PROFILE
APP_PROFILE=prod
```

Two commands, two answers for the same container. Which one tells the truth, where does the other come from, and how do you avoid the confusion?

Question 8 [Analysis]

`compose.prod.yaml` sets `restart: unless-stopped` on all three services. On a Linux server with Docker, what happens after the machine reboots? On your WSL with rootless Podman, after `wsl --shutdown`? Explain the difference and what to use instead with Podman.

Question 9 [Understanding]

`podman compose up -d --scale api=2` creates `stack_api_1` and `stack_api_2`. How does `web` spread requests between the two, with no configuration at all? Why would `--scale web=2` fail? And what must you do before `podman compose down` with podman-compose?

Question 10 [Diagnosis]

```
$ podman compose up -d
Error: creating container storage: the container name "stack_db_1" is already in use by
3e5c2093…
```

Where does that container come from, why does `up` not reuse it, and what is the repair sequence that keeps the volume?

Question 11 [Analysis]

In `compose.yaml`, the database gets its password through `secrets: [db_password]` and `POSTGRES_PASSWORD_FILE`, the secret being declared as `file: ./db_password.txt`. Compare with `environment: POSTGRES_PASSWORD: S3cr3t`: where does the value appear in each case (`podman compose config`, `podman inspect`, Git)? What must be done with `db_password.txt`, and what does `external: true` change?

Question 12 [Analysis]

Match each element of our `compose.yaml` to its Kubernetes equivalent, one line each: a `service` with `image`, `ports`, the `pgdata` volume, the `healthcheck`, the `depends_on`, the implicit network, `env_file`, `secrets`. Which one has **no** direct equivalent, and why?

Question 13 [Analysis]

The team proposes deploying production with `podman compose -f compose.yaml -f compose.prod.yaml up -d` on a single VM. In which cases is that a reasonable decision? Name four things this solution does not provide, and the signal that says it is time to move to an orchestrator.